

Project Checkpoint 2

Important Note

The original, full dataset can be found here:

<https://cds.climate.copernicus.eu/datasets/reanalysis-era5-single-levels-monthly-means?tab=overview> using the API call attached at the bottom of this notebook

This dataset is massive, thus we are using a significantly smaller subset to perform analysis on. The subset can be found on the GitHub page here:

https://github.com/01Experi/CMSC320_SemesterProject inside the "Checkpoint_Submissions/Checkpoint_2" subdirectory with the title "weather.csv".

This notebook follows the subset data stored in the "weather.csv" dataset. At the bottom of this notebook includes the program created to take the original dataset and turn it into its current smaller form. That program takes a long time to run due to the sheer size of the original dataset and so it is highly recommended that it is not run. The program can also be found on the GitHub.

There is one other CSV file used in this notebook, "cities.csv." This is a self-created dataset used for the purpose of selecting weather data from certain cities. This file does not have to be examined as the entire dataset (only 30 entries) is printed below during Data Cleaning when imported, but also feel free to look it over as it is located in the same folder as "weather.csv".

Data Cleaning

Step 1.1: Import necessary libraries

```
In [1]: import pandas as pd ## Data handling
```

Step 1.2: Import dataset from CSV file

```
In [2]: weather_df = pd.read_csv("weather.csv")

weather_df.head()
```

```
Out[2]:
```

	time	latitude	longitude	number	step	depthBelowLa
0	2021-01-01 06:00:00	35.75	139.75	0	0 days	0.0
1	2021-01-01 12:00:00	35.75	139.75	0	0 days	0.0
2	2021-02-01 06:00:00	35.75	139.75	0	0 days	0.0
3	2021-02-01 12:00:00	35.75	139.75	0	0 days	0.0
4	2021-03-01 06:00:00	35.75	139.75	0	0 days	0.0

Step 1.3: Keep important columns, discard the rest.

```
In [3]: weather_df = weather_df[['time', 'latitude', 'longitude', 'stl1',
'slt', 'sp', 'sd', 'u10', 'v10', 't2m', 'd2m', 'sf', 'e', 'ro',
'tp']]
```

Step 1.4: Display data types of **weather_df** columns.

```
In [4]: weather_df.dtypes
```

```
Out[4]: time          str  
        latitude    float64  
        longitude    float64  
        stl1         float64  
        slt          float64  
        sp           float64  
        sd           float64  
        u10          float64  
        v10          float64  
        t2m          float64  
        d2m          float64  
        sf           float64  
        e            float64  
        ro           float64  
        tp           float64  
        dtype: object
```

Step 1.5: Note that the 'time' column is stored as a string. We will convert it to a datetime object and only store the day, month, and year for convenience.

```
In [5]: weather_df['time'] = weather_df['time'].apply(
        lambda x : pd.to_datetime(x[0:10], format='%Y-%m-%d')
        )

weather_df.head()
```

Out[5]:

	time	latitude	longitude	stl1	slt	sp	
0	2021-01-01	35.75	139.75	NaN	NaN	NaN	N
1	2021-01-01	35.75	139.75	279.91925	3.0	101900.016	C
2	2021-02-01	35.75	139.75	NaN	NaN	NaN	N
3	2021-02-01	35.75	139.75	281.76420	3.0	101633.664	C
4	2021-03-01	35.75	139.75	NaN	NaN	NaN	N

Step 1.6: Due to the way the data was recorded, each location and time has two entires. One for ground data, and the other for air (more specifically: percipitation related data). For each entry, we will combine it with its pair to remove missing data.

```
In [6]: weather_df = weather_df.groupby(['time', 'latitude',  
    'longitude']).max().reset_index()  
  
weather_df.head()
```

Out[6]:

	time	latitude	longitude	stl1	slt	sp	
0	2021-01-01	-90.00	0.00	232.94269	1.0	68938.016	10
1	2021-01-01	-77.75	166.75	267.34894	0.0	96918.016	4.
2	2021-01-01	-37.75	145.00	292.26690	4.0	100400.016	0.
3	2021-01-01	-34.50	301.50	298.18097	0.0	101219.016	0.
4	2021-01-01	-34.00	18.50	295.90167	1.0	100912.016	0.

Step 1.7: Rename columns for clarity.

```
In [7]: weather_df.columns = ['date', 'latitude', 'longitude',
    'soil_temperature_level_1', 'soil_type', 'surface_pressure',
    'snow_depth',
    'u_wind_10m', 'v_wind_10m', 'temperature_2m',
    'dewpoint_temperature_2m', 'snowfall', 'evaporation', 'runoff',
    'total_percipitation']

weather_df.head()
```

```
Out[7]:
```

	date	latitude	longitude	soil_temperature_level_1	soil_type
0	2021-01-01	-90.00	0.00	232.94269	1.0
1	2021-01-01	-77.75	166.75	267.34894	0.0
2	2021-01-01	-37.75	145.00	292.26690	4.0
3	2021-01-01	-34.50	301.50	298.18097	0.0
4	2021-01-01	-34.00	18.50	295.90167	1.0

Step 1.8: In order to add location name data to the dataset, we will use *cities.csv* which maps latitude and longitude to a city/location. First we need to import it into a dataframe.

```
In [8]: city_df = pd.read_csv('cities.csv')

city_df
```

```
Out[8]:
```

	location	continent	latitude	longitude
0	Tokyo, Japan	Asia	35.75	139.75
1	Singapore, Singapore	Asia	1.25	103.75
2	Dubai, UAE	Asia	25.25	55.25
3	Beijing, China	Asia	40.00	116.50
4	Mumbai, India	Asia	19.00	73.00

5	Seoul, South Korea	Asia	37.50	127.00
6	Bangkok, Thailand	Asia	13.75	100.50
7	New York City, NY	North America	40.75	286.00
8	Los Angeles, CA	North America	34.00	241.75
9	Chicago, IL	North America	42.00	272.25
10	Mexico City, Mexico	North America	19.50	260.75
11	Miami, FL	North America	25.75	279.75
12	Vancouver, Canada	North America	49.25	237.00
13	Sao Paulo, Brazil	South America	-23.50	313.25
14	Buenos Aires, Argentina	South America	-34.50	301.50
15	Lima, Peru	South America	-12.00	283.00
16	Rio de Janeiro, Brazil	South America	-23.00	316.75
17	Bogota, Colombia	South America	4.75	286.00
18	Cairo, Egypt	Africa	30.00	31.25
19	Lagos, Nigeria	Africa	6.50	3.50
20	Nairobi, Kenya	Africa	-1.25	36.75
21	Cape Town, South Africa	Africa	-34.00	18.50
22	London, United Kingdom	Europe	51.50	0.00
23	Paris, France	Europe	48.75	2.25
24	Moscow, Russia	Europe	55.75	37.50

25	Sydney, Australia	Oceania	-33.75	151.25
26	Melbourne, Australia	Oceania	-37.75	145.00
27	Perth, Australia	Oceania	-32.00	115.75
28	McMurdo Station, Antarctica	Antarctica	-77.75	166.75
29	Amundsen-Scott, Antarctica	Antarctica	-90.00	0.00

Step 1.9: Use **pandas** `merge()` function to combine the two dataframes on latitude and longitude.

```
In [9]: full_df = weather_df.merge(city_df, how='left', on=['longitude',
'latitude'])

full_df.head()
```

Out[9]:

	date	latitude	longitude	soil_temperature_level_1	soil_type
0	2021-01-01	-90.00	0.00	232.94269	1.0
1	2021-01-01	-77.75	166.75	267.34894	0.0
2	2021-01-01	-37.75	145.00	292.26690	4.0
3	2021-01-01	-34.50	301.50	298.18097	0.0
4	2021-01-01	-34.00	18.50	295.90167	1.0

Step 1.10: From looking at the dtypes earlier, we note that `soil_type` is stored as a float64 despite containing int (whole number) data. `soil_type` is categorical data stored through ordinal encoding. While we are at it, we will reorder the column order to make things more ordered and grouped together visually.

```
In [10]: full_df['soil_type'] = full_df['soil_type'].astype('int64')

full_df = full_df[['location', 'continent', 'date', 'latitude',
'longitude', 'temperature_2m', 'surface_pressure',
'dewpoint_temperature_2m',
'v_wind_10m', 'u_wind_10m', 'evaporation', 'runoff',
'total_precipitation', 'snowfall', 'snow_depth',
'soil_temperature_level_1', 'soil_type']]

full_df.head()
```

```
Out[10]:
```

	location	continent	date	latitude	longitude	temperat
0	Amundsen-Scott, Antarctica	Antarctica	2021-01-01	-90.00	0.00	244.2346
1	McMurdo Station, Antarctica	Antarctica	2021-01-01	-77.75	166.75	268.3069
2	Melbourne, Australia	Oceania	2021-01-01	-37.75	145.00	290.4573
3	Buenos Aires, Argentina	South America	2021-01-01	-34.50	301.50	296.2405
4	Cape Town, South Africa	Africa	2021-01-01	-34.00	18.50	294.8030

Step 1.11: Before saving this dataframe into a file, we will first check that everything looks alright.

In
[11]:

```
full_df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 1800 entries, 0 to 1799
Data columns (total 17 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   location                             1800 non-null   str
1   continent                             1800 non-null   str
2   date                                 1800 non-null   datetime64[us]
3   latitude                             1800 non-null   float64
4   longitude                             1800 non-null   float64
5   temperature_2m                       1800 non-null   float64
6   surface_pressure                     1800 non-null   float64
7   dewpoint_temperature_2m              1800 non-null   float64
8   v_wind_10m                           1800 non-null   float64
9   u_wind_10m                           1800 non-null   float64
10  evaporation                           1800 non-null   float64
11  runoff                               1800 non-null   float64
12  total_percipitation                  1800 non-null   float64
13  snowfall                             1800 non-null   float64
14  snow_depth                           1800 non-null   float64
15  soil_temperature_level_1              1800 non-null   float64
16  soil_type                             1800 non-null   int64
dtypes: datetime64[us](1), float64(13), int64(1), str(2)
memory usage: 239.2 KB
```

We notice that there are no null values, and that every column is stored as the desired dtype. Now it is time to perform data exploration on the cleaned dataset.

Data Exploration

Step 2.1: Import additional necessary libraries

In
[12]:

```
from scipy.stats import f_oneway ## For ANOVA test
from scipy.stats import tukey_hsd ## Post-hoc test for ANOVA

import matplotlib.pyplot as plt ## Plotting
import seaborn as sns ## Plotting
```

Step 2.2: Perform first data exploration.

Intro:

One way that we are going to explore the dataset is by looking at how the different variables correlate to each other. Measuring the relationship between the variables can show us which variables are linked. The correlation number ranges from -1 (perfect negative correlation) to +1 (perfect positive correlation).

```

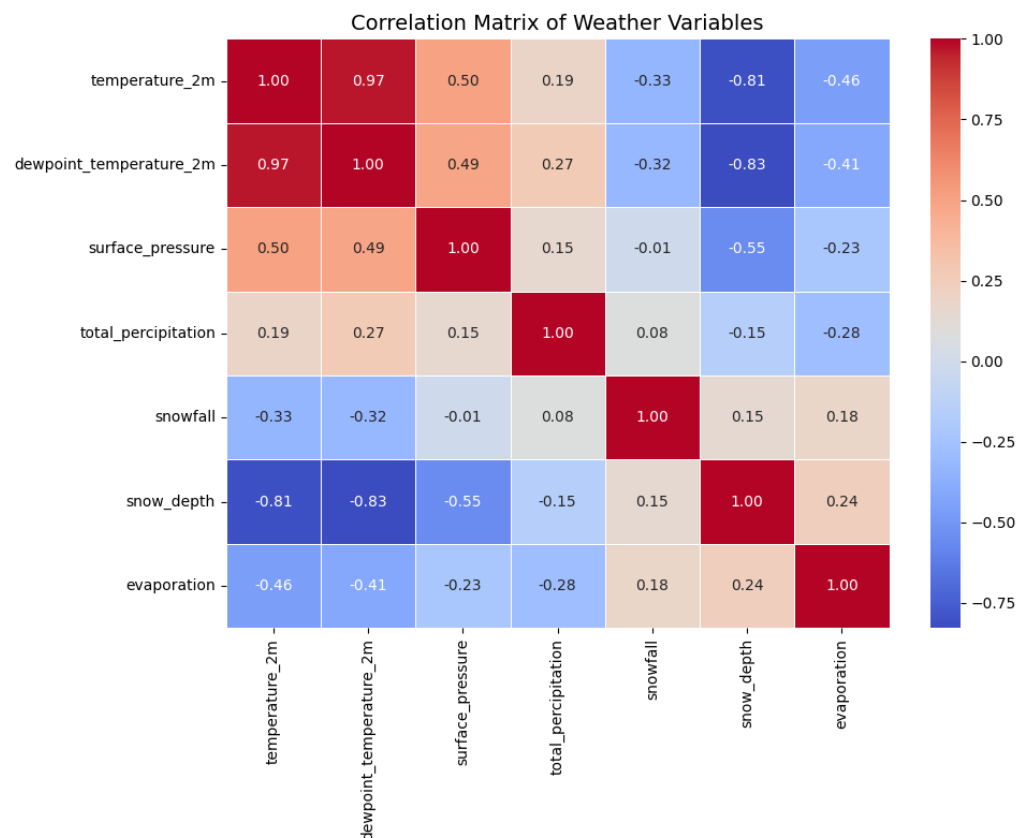
In [13]: df = full_df

corr_cols = [
    "temperature_2m",
    "dewpoint_temperature_2m",
    "surface_pressure",
    "total_percipitation",
    "snowfall",
    "snow_depth",
    "evaporation"
]

corr_matrix = df[corr_cols].corr()

plt.figure(figsize=(10, 8))
sns.heatmap(corr_matrix, annot=True, cmap="coolwarm", fmt=".2f",
            linewidths=0.5)
plt.title("Correlation Matrix of Weather Variables", fontsize=14)
plt.tight_layout()
plt.show()

```



Conclusion:

It is clear that temperature and dewpoint are strongly correlated, .97, which makes sense since warm air holds more moisture, so they rise and fall together. Snow depth and temperature is strongly negatively correlated, -.81, pretty obvious that cold air leads to more snow. Snow depth and dewpoint are also strongly negative, -.83, because drier cold air means more snow sticks. Surface pressure and temperature are moderately positive, .5, since warmer air can create higher pressure systems. And precipitation is weakly correlated with pretty much everything, which is because rain depends on more complex factors, not just temperature.

Step 2.3: Perform second data exploration.

Intro: We can perform quantile analysis to determine how the spread of global temperature statistics is being affected by each continent.

```
In
[14]: df = full_df
temp_quantiles = df['temperature_2m'].quantile([0.0, 0.25, 0.5,
0.75, 1.0])
print("---- Five-Number Summary for Temperature (K) ----")
print(temp_quantiles)

mean = df['temperature_2m'].mean()
median = df['temperature_2m'].median()
print(f"\nGlobal Mean: {mean:.2f}K")
print(f"Global Median (50%): {median:.2f}K")
print(f"Skewness indicator (Mean - Median): {mean - median:.2f}")

grouped_stats = df.groupby('continent')
['temperature_2m'].describe()
print("\n---- Grouped Temperature Statistics by Continent ----")
print(grouped_stats[['mean', '50%', 'std', 'min', 'max']])

plt.figure(figsize=(12, 6))
sns.boxplot(data=df, x='continent', y='temperature_2m')
plt.title('Temperature Distribution by Continent (Quantile
Analysis)')
plt.ylabel('Temperature (Kelvin)')
plt.xlabel('Continent')
plt.grid(axis='y', linestyle='--', alpha=0.3)
plt.show()
```

--- Five-Number Summary for Temperature (K) ---

```
0.00    213.459440
0.25    283.468825
0.50    291.365230
0.75    298.432658
1.00    311.771500
```

Name: temperature_2m, dtype: float64

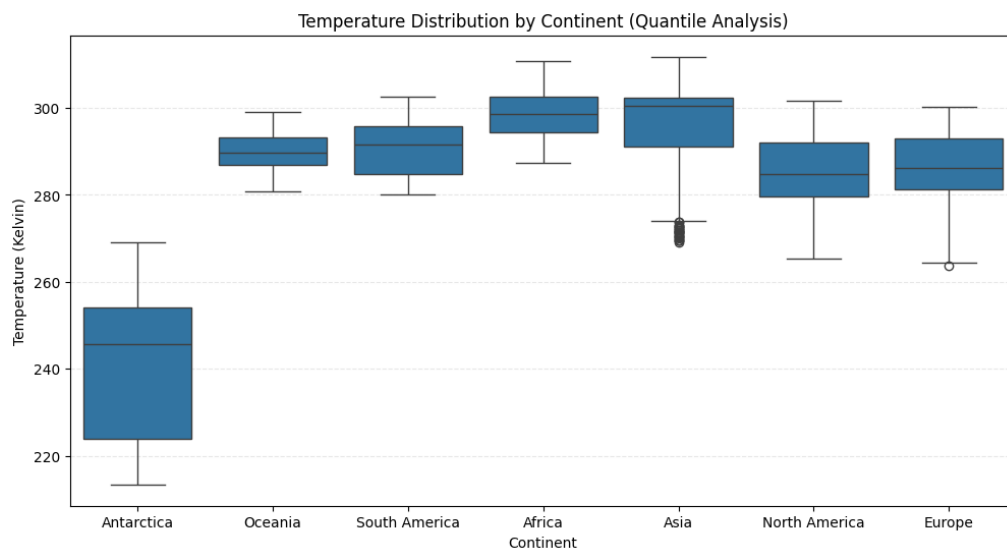
Global Mean: 288.29K

Global Median (50%): 291.37K

Skewness indicator (Mean – Median): -3.08

--- Grouped Temperature Statistics by Continent ---

	mean	50%	std	min	max
continent					
Africa	298.454338	298.651885	5.446771	287.24640	310.70563
Antarctica	241.780144	245.692750	16.364284	213.45944	269.04803
Asia	296.223089	300.549820	10.239304	269.03314	311.77150
Europe	286.253548	286.249695	8.321810	263.63990	300.32767
North America	285.538428	284.784350	8.040009	265.26776	301.52570
Oceania	289.908465	289.777985	4.371169	280.73035	299.07600
South America	291.196757	291.659500	5.516678	280.07648	302.48280



Conclusion: The global temperature distribution is negatively skewed, as the mean is lower than the median. The box plot shows that this is due to how extreme the lows are in Antarctica compared to the rest of the world. Antarctica has a low of 213.46, which is roughly 50 degrees less than the low of the next closest continent, Europe.

Step 2.4: Perform third data exploration.

Intro: We perform a hypothesis test to determine whether the latitude band in which the cities reside have an effect on the total precipitation. We will do this by first running an ANOVA test then following it with a HSD post-hoc test to either support or reject the null hypothesis that total precipitation across latitude bands is the same.

```
In [15]: #Null Hypothesis: Total precipitation across latitude bands is the
        same.
        #Alternative Hypothesis: Total precipitation across latitude bands
        is not the same.

        hypo_df = full_df

        hypo_df['latitude_bands'] = pd.cut(hypo_df['latitude'], bins=[-90,
        -30, 30, 90], labels=['Southern', 'Mid', 'Northern'],
        include_lowest=True)

        hypo_df['total_precipitation_mm'] = hypo_df['total_percipitation']
        * 1000
        rainy_days_df = hypo_df[hypo_df['total_precipitation_mm'] > 0]
        # rainy_days_df.groupby('latitude_bands')
        ['total_precipitation_mm'].describe()

        south = rainy_days_df[rainy_days_df['latitude_bands'] ==
        'Southern']['total_precipitation_mm']
        mid = rainy_days_df[rainy_days_df['latitude_bands'] == 'Mid']
        ['total_precipitation_mm']
        north = rainy_days_df[rainy_days_df['latitude_bands'] ==
        'Northern']['total_precipitation_mm']

        f_statistic, p_value = f_oneway(south, mid, north)

        print(f"P-value: {p_value}")
```

P-value: 7.142553035675129e-11

```
In result = tukey_hsd(south, mid, north)
[16]: print(result)
```

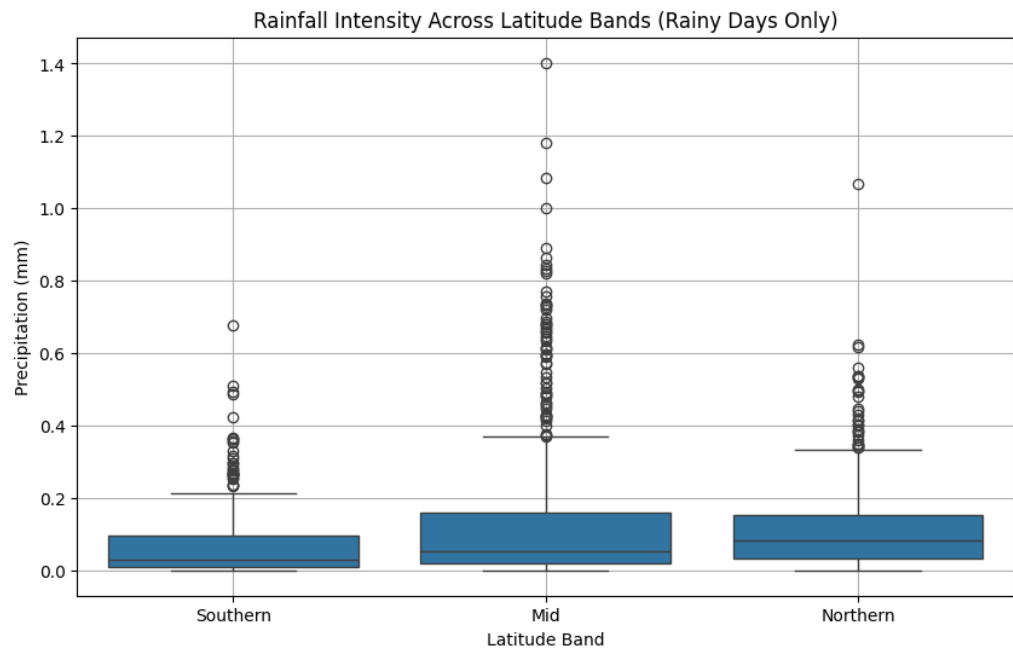
Pairwise Group Comparisons (95.0% Confidence Interval)

Comparison	Statistic	p-value	Lower CI	Upper CI
(0 - 1)	-0.062	0.000	-0.083	-0.040
(0 - 2)	-0.044	0.000	-0.066	-0.022
(1 - 0)	0.062	0.000	0.040	0.083
(1 - 2)	0.018	0.076	-0.001	0.037
(2 - 0)	0.044	0.000	0.022	0.066
(2 - 1)	-0.018	0.076	-0.037	0.001

```
In plt.figure(figsize=(10,6))
[17]: sns.boxplot(data=rainy_days_df, x='latitude_bands',
                y='total_precipitation_mm')

plt.title('Rainfall Intensity Across Latitude Bands (Rainy Days Only)')
plt.xlabel('Latitude Band')
plt.ylabel('Precipitation (mm)')
plt.grid(True)

plt.show()
```



Conclusion: The ANOVA test indicates a significant difference in total precipitation across latitude bands ($p\text{-value} < 0.05$). The Tukey HSD post-hoc test reveals that the Southern band significantly differs from the Northern and Mid bands. However, there is statistically not much difference between the Northern and Mid Bands ($p = 0.076$). This is reflected in the boxplot as the Southern group has lower median rainfall and an overall smaller spread. Thus, we reject the null hypothesis at 5% significance level since the Southern latitude band significantly differs from the other two latitude bands.

Original Dataset Conversion (To Show Work)

The following shows the program used to convert the original dataset into its smaller more manageable form. Do not run unless you have a good amount of RAM, the full dataset on hand, and are willing to wait a long time (like 30 minutes).

```
In [18]: # import pandas as pd
# import sqlite3
# import xarray as xr
# import cfgrib

# if __name__ == '__main__':

#     datasets = cfgrib.open_datasets('era5_subset.grib')
#     ds = xr.merge(datasets, compat='override',
# combine_attrs='drop')

#     # converting to DataFrame object risks memory overload due
# to limited RAM capacity for some computers
#     df = ds.to_dataframe().reset_index()

#     database_file = 'era5_database.sqlite'
#     main_table = 'main'

#     # connect to and create SQL database
#     conn = sqlite3.connect(database_file)

#     df.to_sql(main_table, conn, if_exists='replace',
index=False, chunksize=10000)

#     # select subset from dataset based on selected
# cities/locations
#     city_df = pd.read_csv("cities.csv")
#     s_df = pd.read_sql("SELECT * FROM main LIMIT 0", conn) # get
# column format

#     # for each city get the associated datapoints
#     for _, row in city_df.iterrows(): # get all weather data for
# each location
#         query = f""""SELECT * FROM main WHERE latitude =
# {row['latitude']} AND longitude = {row['longitude']}""""
#         s_df = pd.concat([s_df, pd.read_sql(query, conn)],
# ignore_index=True)

#     # save subset as CSV file
#     s_df.to_csv("weather.csv", index=False)
#     conn.close()
```

API Call (For Original Dataset)

Going to the original dataset linked at the top of this notebook, one can use the following API call to retrieve a subset of the full dataset (this notebook then uses a subset of that subset which is of a larger dataset)

```
dataset = "reanalysis-era5-single-levels-monthly-means"
request = {
  "product_type": ["monthly_averaged_reanalysis_by_hour_of_day"],
  "variable": [
    "10m_u_component_of_wind",
    "10m_v_component_of_wind",
    "2m_dewpoint_temperature",
    "2m_temperature",
    "surface_pressure",
    "total_precipitation",
    "evaporation",
    "runoff",
    "soil_temperature_level_1",
    "soil_type",
    "snowfall",
    "snow_depth"
  ],
  "year": [
    "2021", "2022", "2023",
    "2024", "2025"
  ],
  "month": [
    "01", "02", "03",
    "04", "05", "06",
    "07", "08", "09",
    "10", "11", "12"
  ],
  "time": ["12:00"],
  "data_format": "grib",
  "download_format": "unarchived"
}
```